

# FOUNDATION\_CACHE - Redis Cache Foundation

Developer implementation guide: FOUNDATION\_CACHE\_DEV\_IMPL\_GUIDE.md.

## 1. Purpose

This foundation defines how SOPHIX modules use Redis for non-authoritative cache data.

Use this document when implementing:

- Cache-aside reads.
- Kafka-driven cache refresh or invalidation.
- Redis key naming.
- Cache TTLs.
- Cache reconciliation and invalidation endpoints.

## 2. Ownership

FOUNDATION\_CACHE owns:

- Redis cluster topology.
- Cache eligibility rules.
- Key naming.
- TTL defaults.
- Cache-aside behavior.
- Kafka-driven cache synchronization.
- Cache recovery operations.

FOUNDATION\_CACHE does not own:

- Authoritative business state.
- Financial ledgers, balances, invoice totals, or subscription status.
- Kafka event contracts. See FOUNDATION\_KAFKA.md.

## 3. Core Principle

Redis is an optimization. PostgreSQL and module APIs remain the source of truth.

If Redis is unavailable, the module must call the source-of-truth API and continue.

## 4. Cluster Topology

| Item               | Requirement                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------------|
| Mode               | Redis Cluster                                                                                                       |
| Shards             | 2                                                                                                                   |
| Replicas per shard | 1                                                                                                                   |
| Total nodes        | 4                                                                                                                   |
| Persistence        | AOF enabled                                                                                                         |
| Auth               | ACL plus TLS                                                                                                        |
| Endpoint list      | redis-1.sophix.internal:6379,redis-2.sophix.internal:6379,redis-3.sophix.internal:6379,redis-4.sophix.internal:6379 |

Redis is not behind NGINX. Clients connect to the cluster nodes and follow MOVED / ASK redirects.

Availability rule: this topology is designed to survive one node failure per shard. If a primary fails, its replica is promoted. If a primary and its replica for the same shard are both lost, that shard is unavailable and modules must fall back to source APIs.

## 5. What May Be Cached

| Data                                                    | Cache?         | Reason                             |
|---------------------------------------------------------|----------------|------------------------------------|
| External catalog data consumed by another module        | Yes            | Read-heavy, changes through events |
| Service/package definitions read by fulfillment/billing | Yes            | Safe with event invalidation       |
| Pricing reference data                                  | Yes, short TTL | Read-heavy but changes matter      |
| Session-like temporary data                             | Yes            | Volatile by design                 |
| A module's own source-of-truth tables                   | No             | Read local PostgreSQL directly     |
| Subscription status                                     | No             | Mutable operational state          |
| Wallet balance                                          | No             | Money state                        |
| Invoice totals and postings                             | No             | Strong consistency required        |
| Secrets or tokens                                       | No             | Use secret manager                 |

Rule: a module never caches data it owns.

## 6. Key Naming

Pattern:

```
sophix:{sourceModule}:{aggregate}:{id}
```

Examples:

```
sophix:plm:service:svc_01HX
sophix:sip:package:pkg_01HX
sophix:rlm:homepass:hp_01HX
sophix:ilm:customer:cust_01HX
```

Rules:

| Rule        | Description                                                            |
|-------------|------------------------------------------------------------------------|
| CACHE-KEY-1 | Keys are lowercase except IDs where the platform ID is case-sensitive. |
| CACHE-KEY-2 | Prefix identifies the source module, not the caching module.           |
| CACHE-KEY-3 | Do not include PII in keys.                                            |
| CACHE-KEY-4 | Use stable aggregate IDs, not display names.                           |

## 7. TTL Defaults

| Data type                    | TTL                                       |
|------------------------------|-------------------------------------------|
| Catalog reference data       | 24 hours                                  |
| Pricing                      | 1 hour                                    |
| Volatile lookups             | 5 to 15 minutes                           |
| User/session temporary state | Domain-specific, usually less than 1 hour |

Kafka-driven invalidation is primary. TTL is only a safety net.

## 8. Cache-Aside Read Pattern

```
flowchart TD
    A[Need external data] --> B[GET Redis key]
    B -->|Hit| C[Return cached value]
    B -->|Miss| D[Call source REST API]
    B -->|Redis error| D
    D --> E{Source API success?}
    E -->|Yes| F[SET Redis key with TTL]
    E -->|No| H[Return source error]
    F --> G[Return value]
```

Implementation rules:

| Rule         | Description                         |
|--------------|-------------------------------------|
| CACHE-READ-1 | Treat Redis errors as cache misses. |

| Rule         | Description                                                               |
|--------------|---------------------------------------------------------------------------|
| CACHE-READ-2 | Never fail a business request only because Redis is down.                 |
| CACHE-READ-3 | Fetch from source API on miss.                                            |
| CACHE-READ-4 | Cache only successful source responses.                                   |
| CACHE-READ-5 | Always set TTL.                                                           |
| CACHE-READ-6 | Validate cached JSON version if the source payload has schema versioning. |

## 9. Kafka-Driven Synchronization

When a source module emits a create/update/retire event, consuming modules update or evict cached values through their inbox worker.

Example:

```
{
  "eventType": "ServiceUpdated",
  "payload": {
    "serviceId": "svc_01HX"
  }
}
```

Valid strategies:

Eager refresh:

```
GET /api/services/svc_01HX from PLM
SET sophix:plm:service:svc_01HX <json> EX 86400
```

Lazy evict:

```
DEL sophix:plm:service:svc_01HX
```

Retire/delete events must evict.

If Redis is unavailable during event handling, the inbox row must remain unprocessed and retry later. This keeps cache sync recoverable.

## 10. Redis Command Examples

Simple value:

```
GET sophix:plm:service:svc_01HX
SET sophix:plm:service:svc_01HX "{\"id\":\"svc_01HX\"}" EX 86400
DEL sophix:plm:service:svc_01HX
```

Multi-key fetch:

```
MGET sophix:plm:service:svc_01HX sophix:plm:service:svc_02AB
```

Hash for temporary grouped attributes:

```
HSET sophix:onboarding:session:sess_01HX status "AWAITING_PAYMENT" amount "60000"
EXPIRE sophix:onboarding:session:sess_01HX 3600
HGETALL sophix:onboarding:session:sess_01HX
HINCRBY sophix:onboarding:session:sess_01HX retryAttempts 1
```

## 11. Admin Operations

Every module that uses Redis should expose:

| Endpoint                                          | Purpose                                      |
|---------------------------------------------------|----------------------------------------------|
| POST /admin/cache/invalidate                      | Delete specific keys or a prefix             |
| GET /admin/cache/stats?keyPrefix=...              | Return hit/miss/eviction counters            |
| GET /admin/cache/reconcile?keyPrefix=...&sample=N | Compare sampled cache entries against source |

Invalidate specific IDs:

```
POST /admin/cache/invalidate
Authorization: Bearer <admin-jwt>
Content-Type: application/json

{
```

```

    "keyPrefix": "sophix:plm:service:",
    "ids": ["svc_01HX", "svc_02AB"]
  }

```

Invalidate whole prefix:

```

{
  "keyPrefix": "sophix:plm:service:",
  "all": true
}

```

Prefix invalidation must use SCAN, not blocking KEYS.

## 12. Failure Behavior

| Failure                                   | Behavior                                                                                             |
|-------------------------------------------|------------------------------------------------------------------------------------------------------|
| Single Redis replica down                 | No user impact. Restore replica.                                                                     |
| Single Redis primary down                 | Replica promotes; clients follow redirect.                                                           |
| Primary and replica for same shard down   | That shard is unavailable. Modules fall back to source APIs. Restore Redis and let cache repopulate. |
| Full Redis outage                         | Modules fall back to source REST APIs.                                                               |
| Cache drift                               | Invalidate keys or replay source events through Kafka.                                               |
| Redis unavailable during inbox cache sync | Keep inbox row unprocessed and retry.                                                                |

## 13. Monitoring

Alert on:

| Signal                          | Threshold                                   |
|---------------------------------|---------------------------------------------|
| Hit rate by prefix              | Less than 80 percent sustained              |
| Redis primary count             | Less than 2                                 |
| Memory usage                    | More than 75 percent                        |
| Replication lag                 | More than 5 seconds                         |
| AOF write latency               | More than 100 ms                            |
| Client timeout/connect failures | Any sustained failures                      |
| Cache reconciliation drift      | More than 0 mismatches on critical prefixes |

## 14. Medium Budget Infrastructure

This section defines the recommended on-premise Redis starting point for a medium SOPHIX deployment.

### 14.1 Server Dimensions

Required medium baseline:

| Server role            | Count                   | CPU           | RAM          | Disk                 | Notes                                                     |
|------------------------|-------------------------|---------------|--------------|----------------------|-----------------------------------------------------------|
| Redis node             | 4                       | 2-4 vCPU each | 8-16 GB each | 100-200 GB SSD each  | 2 shards x 1 replica. Cache only; not authoritative data. |
| Monitoring/log storage | Shared platform service | 2 vCPU        | 4 GB         | Depends on retention | Used for Redis metrics, slowlog, alerts.                  |

Minimum acceptable start:

4 x Redis nodes: 2 vCPU / 8 GB RAM / 100 GB SSD

Preferred medium start:

4 x Redis nodes: 4 vCPU / 16 GB RAM / 200 GB SSD

Budget rule: keep four Redis nodes. Reduce RAM before reducing node count. Two nodes would remove shard-level HA and should not be used for production.

Placement rules:

- Put each primary and its replica on different physical hosts.
- Avoid placing all Redis nodes on the same virtualization host.
- Keep Redis separate from PostgreSQL primaries.
- Keep Redis private. Do not expose Redis publicly.
- Prefer low-latency network between application servers and Redis nodes.

## 14.2 Recommended OS

Use one operator-standard Linux distribution:

| OS                                   | Recommendation                                           |
|--------------------------------------|----------------------------------------------------------|
| Ubuntu Server 22.04 LTS or 24.04 LTS | Recommended default for medium teams.                    |
| Rocky Linux 9 / RHEL 9               | Use if the operator standardizes on RHEL-family systems. |

OS baseline:

- 64-bit minimal server install.
- No desktop packages.
- chrony or equivalent NTP enabled.
- Security patching through operator maintenance process.
- SSH restricted to admin VPN/network.
- Host firewall enabled.
- Disable transparent huge pages for Redis.
- Set `vm.overcommit_memory=1`.

Recommended sysctl:

```
vm.overcommit_memory = 1
net.core.somaxconn = 1024
```

## 14.3 Disk Partitions

For each Redis node with 100 GB disk:

```
/                25 GB
/var/lib/redis   50 GB
/var/log         10 GB
/tmp             5 GB
free/reserved    remaining
```

For each Redis node with 200 GB disk:

```
/                30 GB
/var/lib/redis   130 GB
/var/log         20 GB
/tmp             10 GB
free/reserved    remaining
```

Partition rules:

- Put AOF/RDB persistence files under `/var/lib/redis`.
- Do not store Redis persistence files on the root filesystem.
- Alert at 75 percent disk usage and treat 90 percent as critical.
- Use SSD.
- Use xfs or ext4.

## 14.4 Redis Runtime Starting Points

Starting settings:

```
appendonly yes
appendfsync everysec
maxmemory-policy noeviction
protected-mode yes
```

```
tls-port 6379
port 0
```

Memory rule:

- Set maxmemory to about 70 percent of system RAM.
- Leave memory for OS, fork overhead, and AOF/RDB operations.

Example for 8 GB RAM node:

```
maxmemory 5gb
```

Example for 16 GB RAM node:

```
maxmemory 11gb
```

Use noeviction for predictable failure. If Redis is full, writes fail and modules should fall back or evict deliberately. Do not silently evict arbitrary keys that may hide cache sizing problems.

## 14.5 Network Ports

| Flow              | Port                | Exposure                          |
|-------------------|---------------------|-----------------------------------|
| App to Redis TLS  | 6379                | Internal application network only |
| Redis cluster bus | 16379 by convention | Redis nodes only                  |
| Admin SSH         | 22                  | Admin VPN/network only            |
| Monitoring scrape | Operator standard   | Monitoring network only           |

## 15. AWS Deployment Recommendations

This section applies only when deploying Redis-compatible cache on AWS.

### 15.1 Recommended AWS Shape

Preferred managed option:

| Component                         | AWS service                                | Recommendation                                                                                |
|-----------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------|
| Redis cache                       | Amazon ElastiCache for Redis OSS or Valkey | Recommended default for SOPHIX cache use cases.                                               |
| Durable Redis-compatible database | Amazon MemoryDB                            | Use only if a future DD needs Redis-compatible durable primary storage. Not needed for cache. |
| Subnets                           | Private subnets                            | Place nodes in private subnets across multiple Availability Zones.                            |
| Security                          | Security groups + encryption               | Enable encryption in transit and at rest where supported.                                     |
| Monitoring                        | CloudWatch                                 | Monitor memory, CPU, evictions, connections, replication lag, failover.                       |

For SOPHIX cache, ElastiCache is the right default. MemoryDB is overkill because SOPHIX treats Redis as non-authoritative.

### 15.2 ElastiCache Sizing

Medium budget start:

```
Engine:      Redis OSS or Valkey
Cluster mode: enabled
Shards:      2
Replicas/shard: 1
Node type:   cache.m7g.large or cache.m6g.large
```

Lower-cost start if memory demand is still small:

```
Shards:      2
Replicas/shard: 1
Node type:   cache.t4g.medium
```

Preferred medium:

```
Shards:      2-3
Replicas/shard: 1
Node type:   cache.r7g.large or cache.m7g.large
```

Use Graviton node families where available in the target region. Use x86 equivalents only if the operator has a compatibility constraint.

## 15.3 Multi-AZ and Failover

Enable:

- Multi-AZ.
- Automatic failover.
- At least one replica per shard in a different Availability Zone.
- TLS in transit.
- Auth token or user-group ACL.

AWS note: ElastiCache Redis OSS Multi-AZ and AOF are not used together in the same way as self-managed Redis. For SOPHIX this is acceptable because cache is non-authoritative and can be rebuilt from source APIs and Kafka events.

## 15.4 AWS Network Layout

Recommended VPC layout:

Private app subnets:  
- SOPHIX services

Private cache subnets:  
- ElastiCache nodes

Public subnets:  
- No Redis resources

Security group rules:

| Source                | Destination             | Port        | Purpose                 |
|-----------------------|-------------------------|-------------|-------------------------|
| App service SG        | ElastiCache SG          | 6379        | Redis TLS client access |
| Admin VPN/bastion/SSM | Redis client tools host | 22 or SSM   | Admin tool access       |
| Monitoring            | CloudWatch              | AWS-managed | Metrics and alarms      |

Do not expose Redis publicly.

## 15.5 AWS Monitoring

Alarm on:

| Signal                | Threshold                         |
|-----------------------|-----------------------------------|
| Engine CPU            | Sustained above 75 percent        |
| Database memory usage | Above 75 percent                  |
| Evictions             | More than 0 for critical prefixes |
| Replication lag       | Sustained increase                |
| CurrConnections       | Unexpected spike                  |
| Cache hit rate        | Below 80 percent sustained        |
| Failover event        | Any                               |

Reference docs:

- [ElastiCache Multi-AZ for Redis OSS/Valkey](<https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/AutoFailover.html>)
- [ElastiCache supported node types](<https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/CacheNodes.SupportedTypes.html>)
- [Amazon MemoryDB documentation](<https://docs.aws.amazon.com/memorydb/>)

## 16. Developer Checklist

- [ ] Confirm the module does not cache its own source-of-truth data.

- [ ] Key follows `sophix:{sourceModule}:{aggregate}:{id}`.
- [ ] No PII in the key.
- [ ] TTL is set on every cached value.
- [ ] Redis failure falls back to source API.
- [ ] Kafka update/retire events refresh or evict the cache.
- [ ] Inbox retry protects cache sync from Redis outages.
- [ ] Admin invalidation endpoint exists for important prefixes.
- [ ] Financial and mutable operational state is not cached.